



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

8

Classes de base en Java

8- Classes de base

8.1 Classe Object (1/5)



- La classe **Object** est la classe mère de toutes les classes.
 - Elle contient plusieurs méthodes que toutes les classes Java peuvent redéfinir telles que :
 - **public String toString()** : qui retourne une description de l'objet.
 - **public boolean equals (Object)** : qui permet de tester l'égalité sémantique de deux objets.
 - **protected Object clone()** : qui permet de créer et retourner une copie de l'objet courant.
 - La classe **Object** contient également la méthode **getClass()** qui retourne un objet, instance d'une classe particulière appelée **Class**. Cette méthode peut être utilisée sur tout objet pour déterminer sa nature.
 - nom de la classe,
 - nom de ses ancêtres,
 - structure.

8- Classes de base

8.1 Classe Object (2/5)

POO : Java



Sofiane HACHICHA
ISIMG 2025 - 2026

● Exemple de manipulation de la classe **Object** :

```
class Personne // dérive de Object par défaut
{ private String nom;
  public Personne(String nom) { this.nom = nom; }
  public String toString() { return "Classe : " + getClass().getName() + " Objet : " + nom; }
  public boolean equals(Object obj) {
    if (this == obj) return true;
    else if ( obj instanceof Personne p ) return p.nom.equals(this.nom);
    else return false; }
  public int hashCode() { return java.util.Objects.hash(nom); }
}
```

Exemple d'utilisation

```
Personne p1 = new Personne("Foulen");
Personne p2 = new Personne("Foulen");
System.out.println(p1);
System.out.println(p1==p2) ;
System.out.println(p1.equals(p2));
```

Résultat d'affichage

```
Classe : Personne  Objet : Foulen
false
true
```





8- Classes de base

8.1 Classe Object (3/5)

```
public class Simple{  
    String nom;  
    public Simple(String nom) {  
        this.nom = nom;  
    }  
}
```

```
public class TestSimple{  
    public static void main(String[] args){  
        Simple obj1 = new Simple("abc");  
        System.out.println(obj1.nom);  
  
        Simple obj2=obj1;  
        obj2.nom= "xyz";  
        System.out.println(obj1.nom);  
    }  
}
```

Résultat d'affichage

abc

xyz

- **obj1 et obj2 désignent le même objet**
- **Comment dupliquer un objet ?**



8- Classes de base

8.1 Classe Object (4/5)



- L'intérêt de cloner un objet : avoir deux versions, d'un même objet, susceptibles d'**évoluer différemment**.
- Pour réaliser le **clonage** d'un objet, il faut :
 - implémenter l'interface marqueur **Cloneable**
 - déclarer **public** la méthode **clone()** et la redéfinir
 - appeler (systématiquement) **super.clone()**;

```
class Simple implements Cloneable {  
    String nom;  
    public Simple(String nom) {  
        this.nom = nom;  
    }  
    @Override  
    public Object clone() throws CloneNotSupportedException {  
        return (Simple) super.clone();  
    }  
}
```





8- Classes de base

8.1 Classe Object (5/5)

P.O.O : Java



```
public class TestSimple {  
  
    public static void main(String[] args) throws  
        CloneNotSupportedException{  
  
        Simple obj1 = new Simple("abc");  
        System.out.println(obj1.nom);  
  
        Simple obj2=(Simple)obj1.clone();  
  
        obj2.nom= "xyz";  
        System.out.println(obj1.nom);  
  
    }  
}
```

Résultat d'affichage

abc

abc



8.2 Chaînes de caractères non modifiables (1/4)



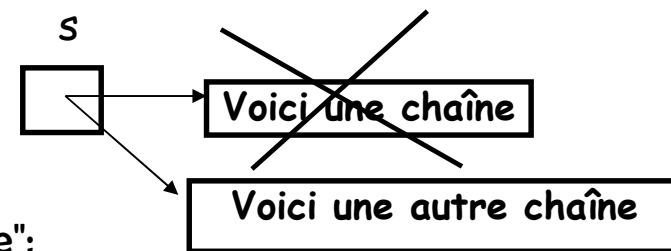
● La classe **String** :

- La classe **String** décrit des objets qui contiennent une chaîne de caractères **constante** (cette chaîne ne peut pas être modifiée).

String s ;

s = "voici une chaîne";

s = "voici une autre chaîne";



- La classe String est **immuable** car ces objets sont immuables.
 - Un objet est immuable si une fois qu'il est créé, il est impossible de changer son état.
- String est la seule classe qui dispose d'opérateurs (+ et +=) pour la concaténation de chaînes de caractères.

String message = "Mon nombre est " + 3 ;

8.2 Chaînes de caractères non modifiables (2/4)



- Une chaîne de caractères multi-lignes « **Text Block** » s'introduit via `"""` et se termine par la même séquence de caractères. Un retour à la ligne est imposé après la séquence de caractères d'ouverture du Text Block.

- Les Text Blocks sont disponibles depuis **Java 15**.
- Il est possible d'indenter le contenu d'un Text Block.

```
String str = """
```

```
    Bonjour
```

```
    CPI2 """;
```

```
System.out.println(str);
```

```
Bonjour
    CPI2
```





8.2 Chaînes de caractères non modifiables (3/4)



● Classe **java.lang.String** :

● Quelques méthodes utiles de **String** :

- **int** `length()` : retourne le nombre de caractères dans la chaîne.
- **char** `charAt(int index)` : retourne le caractère à la position **index** spécifiée dans la chaîne.
- **int** `indexOf(String s, int i)` : retourne la position de la première occurrence de la chaîne **s** en partant de la position **i**
- **String** `substring(int i, int j)` : retourne une chaîne extraite de la chaîne initiale en partant de la position **i** jusqu'à la position **j-1**.
- **String** `concat(String autre)` : concatène la chaîne avec une **autre**.
- **boolean** `equalsIgnoreCase(String autre)` : compare la chaîne à une **autre** en ignorant la casse.
- **String** `trim()` : supprime les espaces blancs de début et de fin de chaîne.





8- Classes de base

8.2 Chaînes de caractères non modifiables (4/4)

P.O.O : Java



- **String** `toLowerCase( )`, **String** `toUpperCase( )` : majuscule vers minuscule, minuscule vers majuscule.
- **int** `compareTo(String autre)` : compare la valeur lexicale (Unicode) d'une chaîne à une **autre**.
- **String** `format(String format, Object ... args)` : méthode statique qui renvoie une chaîne formatée en utilisant les paramètres fournis. `%d` : permet d'injecter un entier, `%f` : permet d'injecter un flottant, `%c` : permet d'injecter un char, `%s` : permet d'injecter un **String**, etc.

- `// on prend 2 décimales`

```
String str = String.format("Valeur = %.2f", 89.9681024);
```

- `int num = 20, den = 11;`

```
String str1 = String.format( "[%d/%d]", num, den );
```

- `String name = "Ali"; int age = 25;`

```
//%1$ premier argument, %2$ deuxième argument
```

```
String str2 = String.format("My name: %1$s, My age: %2$d", name, age);
```

8.3 Les chaînes de caractères modifiables (1/5)



● Classe **java.lang.StringBuffer** :

- La classe **StringBuffer** en Java est semblable à la classe **String**, sauf qu'elle peut être modifiée.
- Un objet de la classe **StringBuffer** se caractérise par deux tailles, qui sont retournées par les méthodes :
 - **int length()** : qui retourne la longueur de la chaîne, c'est-à-dire le nombre total de caractères.
 - **int capacity()** : qui retourne la capacité actuelle du tampon
- Constructeurs de **StringBuffer** :
 - **StringBuffer()** : crée un tampon de chaîne vide avec la capacité initiale de 16.
 - **StringBuffer(String str)** : crée un tampon de chaîne avec la chaîne spécifiée (**str**).
 - **StringBuffer(int capacity)** : crée un tampon de chaîne vide avec la capacité spécifiée (**capacity**).





8- Classes de base

8.3 Les chaînes de caractères modifiables (2/5)

POO : Java



● Quelques méthodes utiles de **StringBuffer** :

- **StringBuffer append(String p)** : ajoute la chaîne **p** spécifiée à la fin du chaîne (la méthode **append()** est surchargée : **append(char)**, **append(boolean)**, **append(int)**, **append(float)**, **append(double)**, etc.).
- **StringBuffer insert(int offset, String p)** : idem, mais en insérant **p** à la position spécifiée par **offset**.
- **StringBuffer reverse()** : inversion des caractères de la chaîne.
- **StringBuffer replace(int i, int j, String str)** : remplace la chaîne de **i** à **j-1** spécifiés par la chaîne **str**.
- **StringBuffer delete(int i, int j)** : supprime la chaîne de **i** à **j-1** spécifiés.
- **String toString()** : permet de convertir l'objet **StringBuffer** en une chaîne de caractères.
- **void setLength(int taille)** : impose une nouvelle **taille** de la chaîne.



8- Classes de base

8.3 Les chaînes de caractères modifiables (3/5)

POO : Java



● Exemple :

```
StringBuffer s = new StringBuffer("Bonjour");  
s.append(" tout le monde").append(8);  
System.out.println("s : "+s);  
System.out.println("longueur : "+s.length());  
System.out.println ("capacité : "+ s.capacity());  
s.insert(2,"hop");  
String s1=s.toString();  
System.out.println("s1 : "+s1);
```

Résultat d'affichage

```
s : Bonjour tout le monde8  
longueur : 22  
capacité : 23  
s1 : Bohopnjour tout le monde8
```

8- Classes de base

8.3 Les chaînes de caractères modifiables (4/5)

P.O.O : Java



● Classe **java.lang.StringBuilder** :

- **StringBuffer** et **StringBuilder** (qui existe depuis **Java 5**) proposent toutes les deux exactement les mêmes méthodes.
 - Si votre buffer est utilisé par **plusieurs threads** en parallèles, il faudra choisir une implémentation synchronisée contre les accès concurrents, à savoir la classe **StringBuffer**.
 - Cependant, si un **seul thread** manipule votre buffer, on est alors dans un context « **thread-safe** », il faudra alors utiliser la classe **StringBuilder**.
 - Cela permettra d'éviter de poser des verrous de synchronisation et donc, les temps de réponse seront meilleurs.





● Classe **java.util.StringTokenizer** :

- Cette classe permet de construire des objets qui savent découper des chaînes de caractères en sous-chaînes.
- Lors de la construction du **StringTokenizer**, il faut préciser la chaîne à découper et, le ou les caractères qui doivent être utilisés pour découper cette chaîne :

● Exemple :

```
String texte = "ab;abcad;ra;y";
```

```
java.util.StringTokenizer st = new java.util.StringTokenizer(texte, ";");
```

```
System.out.println("Nombre de parties : "+st.countTokens());
```

```
while( st.hasMoreTokens() )
```

```
{ String mot = st.nextToken() ;
```

```
System.out.print(mot+" " ) ; }
```

Résultat d'affichage

Nombre de parties : 4
ab abcad ra y





8- Classes de base

8.4 Classe Scanner (1/2)

POO : Java



- Depuis **Java 5**, la lecture des entrées clavier est effectuée via un objet de la classe **Scanner** appartenant au package **java.util**.
- Pour récupérer les données tapées par l'utilisateur à la console, il faut initialiser l'objet Scanner avec l'entrée standard System.in.
Scanner clavier = new Scanner(System.in);
- La classe Scanner contient divers constructeurs surchargés pour accueillir diverses méthodes d'entrée telles que System.in, un fichier, un chemin, un flux d'entrée (InputStream), etc.
- La classe Scanner contient diverses méthodes de récupération de données pour chaque type de base (sauf les char) : **nextLine()** et **next()** pour les **String**, **nextInt()** pour les **int**, **nextDouble()** pour les **double**, **nextBoolean()** pour les **boolean**, etc. Il est possible de demander à l'objet Scanner si des données sont disponibles par les méthodes **hasNextLine()**, **hasNext()**, etc.

String s = clavier.nextLine();

/* La méthode next() lit l'entrée jusqu'à l'espace et place le curseur sur la même ligne après la lecture de l'entrée. En revanche, la méthode nextLine() lit toute la ligne d'entrée jusqu'à la fin de la ligne, y compris les espaces. */



8- Classes de base

8.4 Classe Scanner (2/2)

```
import java.util.Scanner;
class TestScanner {
    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        System.out.println("Entrez votre nom : ");
        String nom = clavier.nextLine();
        System.out.println("Entrez votre age : ");
        int age = clavier.nextInt();
        System.out.println("Entrez votre moyenne : ");
        float moyenne = clavier.nextFloat();
        System.out.println("Vos caractéristiques sont :");
        System.out.println("Nom : " + nom);
        System.out.println("Age : " + age);
        System.out.println("Moyenne : " + moyenne);
        clavier.close();
    }
}
```

8- Classes de base

8.5 Classe ArrayList (1/5)



- **Une collection est un objet qui contient d'autres objets**
 - Un tableau est une collection
- **ArrayList fournit un tableau dynamique**
 - La classe **java.util.ArrayList** est une classe fréquemment utilisée pour gérer des listes (tableaux dynamiques).
 - Un **ArrayList** se comporte comme un tableau, il contient plusieurs objets
 - Il ne peut pas contenir des types primitifs
 - Accède à ses éléments à l'aide d'un index
 - Pas de taille prédéfinie
 - Définit des méthodes pour ajouter ou supprimer un élément

8- Classes de base

8.5 Classe ArrayList (2/5)



● Classe **java.util.ArrayList** :

● Constructeurs de **ArrayList** :

- **ArrayList()** : crée une liste vide.
- **ArrayList(int l)** : crée une liste vide ayant une capacité initiale **l**.
- **ArrayList(Collection c)** : crée une liste à partir d'une collection **c**.

● Quelques méthodes utiles de **ArrayList** :

- **boolean add(Object elt)** : ajoute l'élément **elt** spécifié à la fin de la liste.
- **void add(int index, Object elt)** : place l'élément **elt** spécifié à la position **index** de la liste.
- **Object remove(int index)** : supprime l'élément de la liste à la position **index**.
- **boolean remove(Object elt)** : supprime la première occurrence de l'élément **elt** et retourne **true** (**false** sinon).





8- Classes de base

8.5 Classe ArrayList (3/5)

- **Object** **get** (**int** index) : retourne l'élément à la position **index** sans le supprimer.
- **Object** **set**(**int** index, **Object** elt) : remplace l'élément de la liste se trouvant à la position **index** par l'élément spécifié **elt** et retourne l'élément avant le remplacement.
- **int** **indexOf**(**Object** elt) : retourne la position de la première occurrence de l'élément **elt** (-1 si **elt** n'est pas présent).
- **Iterator**<**Object**> **iterator**() : retourne un itérateur sur la liste.
- **boolean** **contains**(**Object** elt) : retourne **true** si l'élément **elt** est présent dans la liste.
- **boolean** **isEmpty**() : retourne **true** si la liste ne contient pas d'éléments.
- **void** **clear**() : supprime tous les éléments de la liste.
- **int** **size**() : retourne le nombre d'éléments dans la liste.



8- Classes de base

8.5 Classe ArrayList (4/5)

POO : Java



```
import java.util.*;
class Etudiant{
    private String leNom;
    public Etudiant(String unNom){ leNom = unNom; }
    public void setNom(String nom) { leNom = nom; }
    public String getNom() { return leNom; }
}
class TestEtudiant1 { //(utilisant la classe Object)
    public static void main(String[] args) {
        ArrayList <Object> tableauEtudiants= new ArrayList<Object>();
        Etudiant e1 = new Etudiant("Sami");
        Etudiant e2 = new Etudiant("Ali");
        tableauEtudiants.add(e1);
        tableauEtudiants.add(e2);
        if (! tableauEtudiants.isEmpty()) {
            for (int i = 0; i< tableauEtudiants.size(); i++)
                System.out.println(((Etudiant) tableauEtudiants.get(i)).getNom());
            tableauEtudiants.remove(1);
        }
    }
}
```

8- Classes de base

8.5 Classe ArrayList (5/5)

P.O.O : Java



```
class TestEtudiant2 { //(utilisant la classe Etudiant)
    public static void main(String[] args) {
        ArrayList<Etudiant> tableauEtudiants = new ArrayList<>();
        Etudiant emp1 = new Etudiant("Sami");
        Etudiant emp2 = new Etudiant("Ali");
        tableauEtudiants.add(emp1);
        tableauEtudiants.add(emp2);
        if (!tableauEtudiants.isEmpty()) {
            for (int i = 0; i < tableauEtudiants.size(); i++)
                System.out.println((Etudiant) tableauEtudiants.get(i).getNom());
            tableauEtudiants.remove(1);
        }
    }
}
```

Utilisation de Iterator

```
for (Iterator<Etudiant> i=tableauEtudiants.iterator(); i.hasNext(); )
    System.out.println(i.next().getNom());
```

Boucle for-each

```
for (Etudiant e : tableauEtudiants)
    System.out.println(e.getNom());
```

8- Classes de base

8.6 Classe LinkedList



- Comme **ArrayList**, la classe **LinkedList** implémente l'interface **List**.
 - **ArrayList** utilise un **tableau** extensible
 - Elle est **efficace** pour les méthodes **get()** et **set()**.
 - **LinkedList** est implémentée sous forme d'une **liste chaînée**
 - Les performances **d'ajout et de suppression** pour **LinkedList** sont **meilleures** que celles de **ArrayList** et **moins bonnes** pour les méthodes **get()** et **set()**.
- L'API Collections de Java propose d'autres classes telles que **Vector**, **HashSet**, **TreeSet**, **PriorityQueue**, **ArrayDeque**, **HashMap**, **TreeMap**, etc.

